

Instructor Name
Zaheredin S. Mussa

Qualification
BSc. & MSc. in Computer Science

Email
Shifazaheredin@gmail.com
or
Zaheredin.Shifa@amu.edu.et

Address
Arbaminch University Institute of Technology

Course details

- Course title: **Data Structure and Algorithms**
- Course code: **SENG2082**
- Credit hours: **3** ECTS: **5**
- lecture hrs: **2** Lab hrs: **3**
- Tutorial hrs: **0**
- Prerequisite: **SENG1072 - Fundamental Programming II**
- Course category: **Core**
- Year: **II** Semester: **II**

Course objectives

- To introduce the analyze complexity algorithms
- To introduce the most common data structures like linked lists, stacks, queues, trees, and graphs
- To practice sorting, & searching on different DS
- To implement the data structures with a chosen programming language

DATA STRUCTURES AND ALGORITHMS

CHAPTER ONE
Algorithm Analysis Concepts

CHAPTER TWO
Simple Sorting & Searching Algorithm

CHAPTER THREE
Linked Lists

CHAPTER FOUR
Stacks and Queues

CHAPTER FIVE
Trees and Graphs

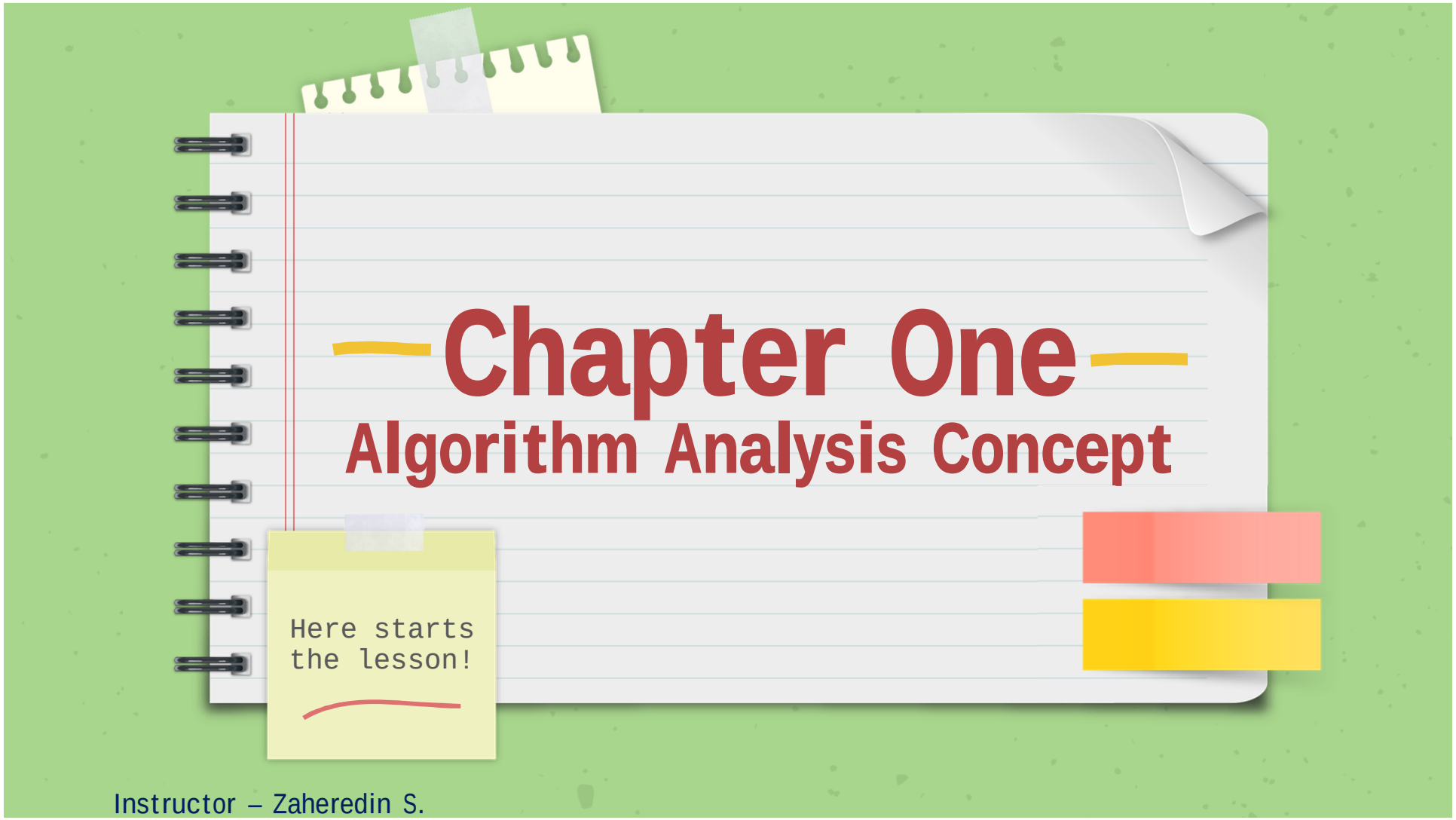
CHAPTER SIX
Advanced Sorting & Searching Algorithms

Text Books / Resources

- Algorithms and Data Structures: The Science of Computing by Baldwin/ Scragg. Charles River Media. 2004.
- Michael T.Goodrich & Roberto Tamassia, “Data Structures and Algorithms in Java”Wiley.

Assessment Methods:

Assessment	% of Final Grade (100)
Assignments	10
Presentation	10
Project	20
Tests	20
Final Exam	40



Instructor – Zaheredin S.

Chapter One Contents!



1.1 Introduction to Algorithm Analysis

Chapter One Contents!



1.2 Asymptotic Analysis

1.1 – Introduction to Algorithm Analysis

What is an "Algorithm"?

- An algorithm is a well-defined procedure or a finite number of steps to be followed for solving a problem.
- This is given in pseudo code format and is independent and implement in any programming language and solved using any type of computers.
- The efficiency of the algorithm plays a major role in determining the efficiency of the program when it is implemented.

Cont...

Main Characteristics of the Algorithm

- **Must take an input:** Zero or more quantities or externally supplied.
- **Must give some output:** At least one quantity must be produced.
- **Definiteness**: Each instruction is clear and unambiguous.
- **Finiteness**: It terminates after a finite number of steps.
- **Effectiveness**: It must be possible to perform each step exactly and in a finite amount of time.
- **Completeness**: It must solve the problem completely.

Cont...

Cont...

- **Language Independence:** It must not depend on any one programming language.
- **Correctness**: It must compute correct answer for all possible legal inputs.
- **Efficiency**: It must solve with the least amount of computational resources such as time and space.

Cont...

Analysis of Algorithm

- **Algorithm analysis** refers to the process of determining the amount of computing time and storage space required by different algorithms.
- An algorithm is mainly analyzed to determine the execution time of the program and how much memory space required.
- Algorithm analysis is required to measure the efficiency of the algorithm and to see now the execution time varies with size of the input.

Cont...

Analysis of algorithm is important for the following reasons:

- Analysis is more reliable than the experimental testing.
- Analysis helps to select better algorithm.
- Analysis identifies the scope of improvement of algorithm.
- Analysis predicts performance.

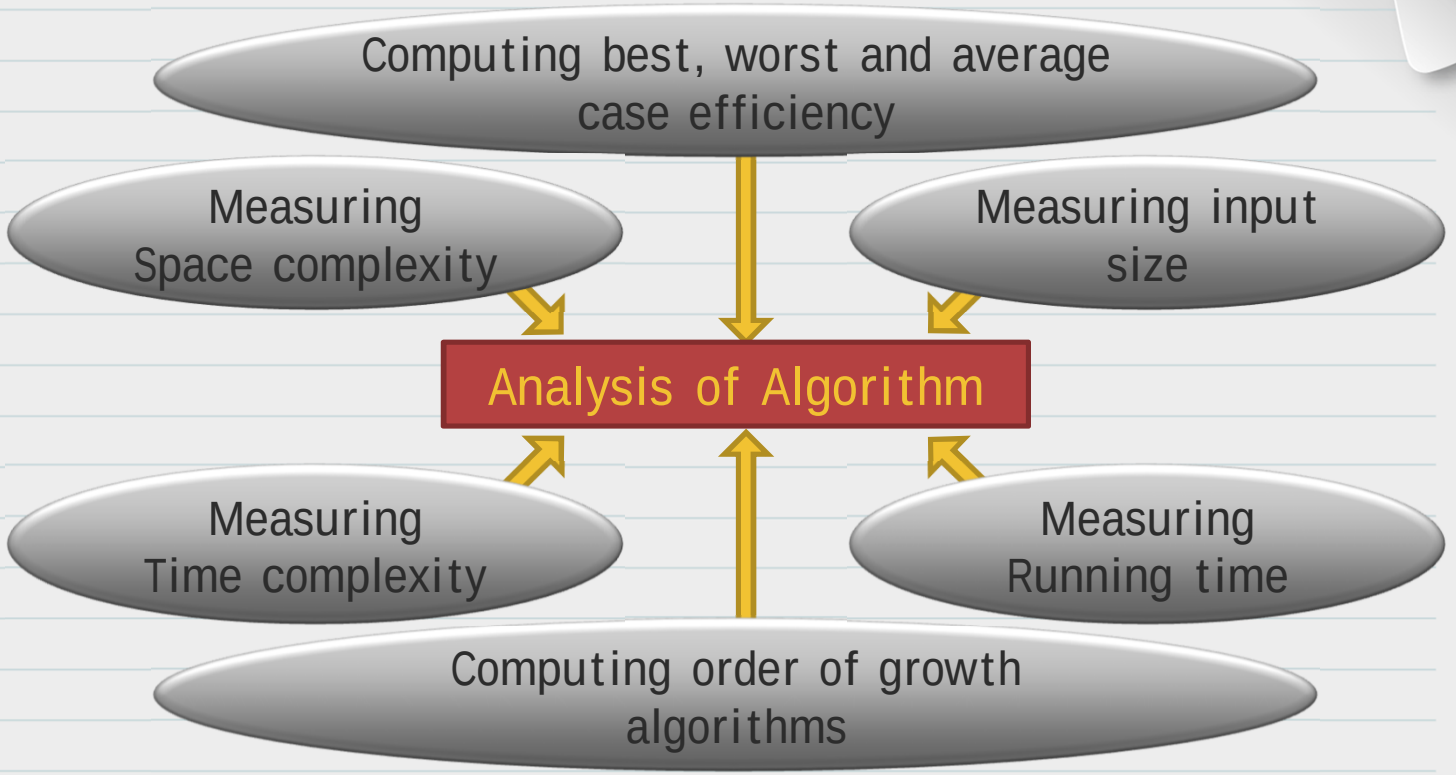
Performance Analysis:

- The efficiency of the algorithm can be decided by measuring the performance of an algorithm.

Cont...

- We can measure the performance of an algorithm by computing two factors:
  I. Amount of space/storage required by an algorithm.
  II. Amount of time required by an algorithm.

Cont...


Computing best, worst and average case efficiency
Measuring Space complexity
Measuring input size
Analysis of Algorithm
Measuring Time complexity
Measuring Running time
Computing order of growth algorithms

Cont...

I. Space Complexity:

- The space complexity of an algorithm P is the amount of memory it requires for running the algorithm.
- Or, it determine the approximate memory required to solve a problem of size n.
- To compute the space complexity we use two factors: constant and instance characteristics.
- It has two parts, i.e a fixed and variable part.

Thus, $S(P) = c + S_P$

Where c is a constant, and S_P is instance characteristics

Cont...

- The **fixed part** component c is independent of the characteristics of inputs and outputs and depends on the space for the code and space for certain type of variables.
- The **variable part** component Sp depends upon the space needed by the component variables, referenced variables and the recursion stack space.
- This recursion stack space includes the space for the formal parameters, local variables and the return address.

Cont...

Algorithm for the sum & product of elements of an array.

```
Algorithm Totprod (a, n)
{
    Tot: =0.0;
    Prod: =1.0;
    For k: =1 to n do {
      Tot: =Tot +a[k];
      Prod: =Prod*a[k];
    }
    return Tot;
    return Prod;
}
```

- In this algorithm the space needed by n, k, Tot and Prod is one word for each and for the array a [ ] is at least n words.

Thus, $S_P = S_{ArrayTot}(n) \geq n+4$

Cont...

Array elements product-A recursive algorithm

```
Algorithm RProd (a, n)
{
  if(n=1) then
    return a[1];
  else
    return RProd(a, n-1)*a[n];
}
```

Cont...

- In this algorithm, the instances are characterized by n.
- The internal stack used for recursion includes space for formal parameters, local variables and return address. The space required by each cell to function.
- Suppose that the return address requires one word of memory. Atleast, three words (space for n, space for the return address and a pointer to a [ ]) are required for each call to RProd.
- Thus, the recursion stack space is greater than or equal to 3(n+1), as depth of recursion is (n+1) (n times call to function and one return call).

Cont...

II. Time Complexity:

- The time complexity of an algorithm is the amount of computer time required by an algorithm to run the completion.
- It is difficult to compute the time complexity in terms of physically clocked time, because clock-time can vary based on many things.
- For instance in multi user system, executing time depends on many factors such as:

  - Number of other programs running
  - System load
  - Speed of underlying hardware.
  - Instruction set used etc.

Cont...

- The time complexity is therefore given in terms of frequency count.
- **Frequency count** is a count denoting number of times of execution of statement.
- Or, it determine the approximate number of operations required to solve a problem of size n.
- The time T(P) taken by the program is sum of the compile time and run time.

Cont...

Analysis Rules:

1. We assume an arbitrary time unit.
2. Execution of one of the following operation takes time 1
   - Assignment Operation
   - Single Input/output Operation
   - Single Boolean Operations
   - Single Arithmetic Operations
   - Function Return
3. Running time of a **selection statement** (if, switch) is the time for the condition evaluation + maximum of the running times for the individual clauses in the selection.

Cont...

4. Running time for a **loop** is equal to running time for the statements inside the loop * number of iterations.
5. The total running time of a statement inside a group of nested loops is the running time of the statements multiplied by the product of the sizes of all the loops. For nested loops, analyze inside out. Always assume that the loop executes the maximum number of iterations possible.
6. Running time of a function call is 1 for setup + the time for any parameter calculations + the time required for the execution of the function body.

Cont...

Examples 1.

```
int count()
{
   int k=0;
   cout<< “Enter an integer”;
   cin>>n;
   for (i=0; i<n; i++)
     k=k+1;
   return 0;
}
```

T (n) = 4n+7

Cont...

Time Units to Compute

1 for the function
1 for the assignment statement: int k=0
1 for the output statement.
1 for the input statement.
In the for loop:
    1 assignment, n+1 tests, and n increments.
    n loops of 2 units for an assignment, and an addition.
    1 for the return statement.

T (n)= 1+1+1+1+(1+n+1+n)+2n+1 = 4n+7

Cont...

Examples 2.

```
int total (int n)
{
    int sum=0;
    for (int i=1;i<=n; i++)
        sum = sum+1;
    return sum;
}
```

T (n) = 4n+5

Cont...

Time Units to Compute

1 for the function

1 for the assignment statement: int sum=0

In the for loop:

- 1 assignment, n+1 tests, and n increments.
- n loops of 2 units for an assignment, and an addition.
- 1 for the return statement.

T (n)=1+1+ (1+n+1+n)+2n+1 = 4n+5

Cont...

Examples 3.

```
voidfunc() {
    int x=0, i=0, j=1;
    cout<< “Enter an Integer value”;
    cin>>n;
    while (i<n){
      x++;
      i++;
    }
    while (j<n){
      j++;
    }
}
```

T (n) = 5n+6

Cont...

Time Units to Compute

1 for the function
1 for the first assignment statement: x=0;
1 for the second assignment statement: i=0;
1 for the third assignment statement: j=1;
1 for the output statement.
1 for the input statement.
In the first while loop:
    n+1 tests
    n loops of 2 units for the two increment (addition) operations

Cont...

In the second while loop:

n tests

n-1 increments

T (n)=1+1+1+1+1+1+n+1+2n+n+n-1 = 5n+6

Cont...

Examples 4.

```
int sum (int n)
{
    int partial_sum = 0;
    for (int i = 1; i <= n; i++)
        partial_sum = partial_sum + (i * i * i);
    return partial_sum;
 }
```

T (n) = 6n+5

Cont...

Time Units to Compute

1 for the function
1 for the assignment.
1 assignment, n+1 tests, and n increments.
n loops of 4 units for an assignment, an addition, and two multiplications.
1 for the return statement.

T (n) =1+1+ (1+n+1+n) +4n+1 = 6n+5

1.2 - Asymptotic Analysis

- To choose the best algorithm, we need to check efficiency of each algorithm.
- The efficiency can be measured by computing time and space complexity of each algorithm.
- **Asymptotic analysis** is the process of calculating the running time of an algorithm in mathematical units.
- In computer programming, the asymptotic analysis tells us the execution time of an algorithm.
- We can calculate the best, average, and worst-case scenarios for an algorithm by using asymptotic analysis.

Cont...

How to find time complexity or running time?

- Computing the real running time of a process is not feasible. The running time to perform any operation depends on the size of the input.
- Example, traversing an array of 5 elements will take less time than traversing through an array of 500 elements.
- From example above, we observe that time complexity depends on the input size.
- **Hence**, if the input size is 'n', then **f(n)** is a function of 'n' denoting the time complexity.

Cont...

Analysis of algorithm is concerned on three cases:

- **Best-case complexity** - it is the function defined by minimum number of steps taken on any instance of size n.
- **Average case complexity** - it is the function defined by the average number of steps taken on any instance of size n.
- **Worst-case complexity** - it is the function defined by the maximum number of steps taken on any instance of size n.

Cont...

Asymptotic Notations

- Asymptotic notations are expressions that are used to calculating the running time complexity of an algorithm.

Commonly used asymptotic notations are:

1. Big oh Notation (Upper bound):

- The Big oh notation is denoted by '**O**'.
- It is a method of representing the upper bound of algorithm's running time.
- Using Big oh notation we can give longest amount of time taken by the algorithm to complete.

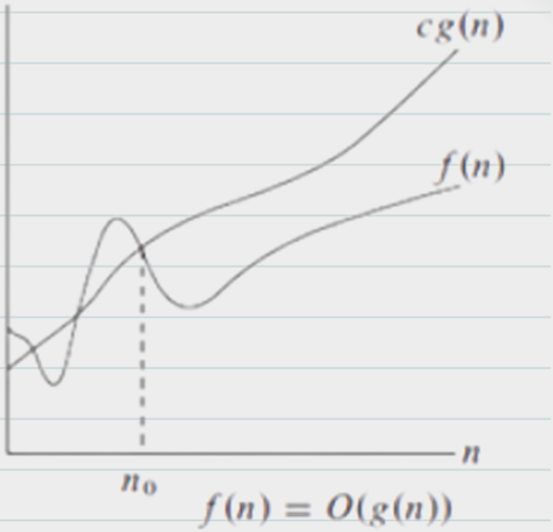
Cont...

Definition:

- Let f(n) and g(n) be two non-negative functions.
- The function **f(n)=O(g(n))** if there exist positive constants c and n_0 such that **f(n)** $\leq$ **c*g(n)** for all n $\geq$ n_0. Then, we can say f(n) is big-oh of g(n).
- It is also denoted as **f(n)** ε **O(g(n))**.
- In other way f(n) is less than g(n) if g(n) is multiple of some constant c.

Cont...

- Big oh (O) notation gives an upper bound for a function to within a constant factor.
- We write **f(n)=O(g(n))** if there are positive constants n_0 and c, such that to the right of n_0, the value of **f(n)** is always lie on or below **c*g(n)**.
- Thus, always upper bound of existing time is obtained by Big oh notation.



Cont...

Example - Prove that $5n^2+3n+1 = O(n^2)$

- **Solution -** Let, $f(n)= 5n^2+3n+1$ and $g(n)=n^2$

  $f(n) \leq c*g(n)$

  $5n^2+3n+1 \leq 5n^2+3n^2+1n^2$

  $5n^2+3n+1 \leq 9n^2$ when $n \geq 1$

  So $n \geq 1$ then $5n^2+3n+1 \leq 9n^2$

- From the definition **$f(n) \leq c*g(n)$ for $n \geq 1$**, Here $c=9$, $n_o=1$ and **$g(n)=n^2$**
- **Hence, $5n^2+3n+1 = O(n^2)$ proved.**

Cont...

2. Omega Notation (Lower bound):

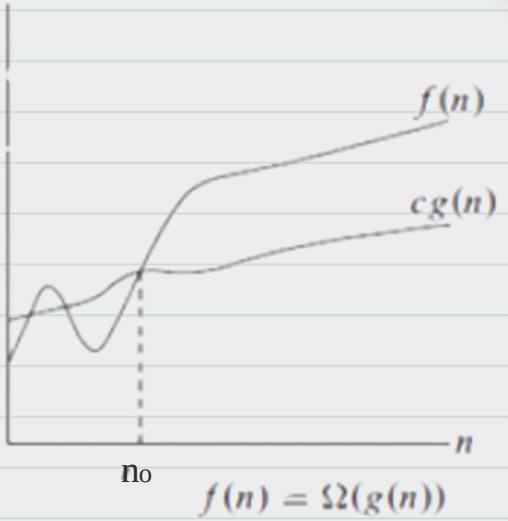
- It is denoted by Ω.
- This notation is used to represent the lower bound of algorithm's running time.
- Using omega notation we can denote shortest amount of time taken by algorithm.

Definition:

- Let f(n) and g(n) be two non-negative functions.
- The function **f(n)=Ω(g(n))** if there exist positive constants c and n_0 such that **f(n)** ≥ **c*g(n)** for all n ≥ n_0.

Cont...

- It is denoted as f(n) ε Ω(g(n)).
- Omega (Ω) notation gives an lower bound for a function to within a constant factor.
- We write **f(n)=Ω(g(n))** if there are positive constants n_0 and c, such that to the right of n_0, the value of **f(n)** is always lie on or above **cg(n)**.



Cont...

Example - Prove that $5n2+3n+1 = \Omega(n^2)$

- **Solution -** Let $f(n)= 5n^2+3n+1$ and $g(n^2)$

  $f(n) \geq c*g(n)$

  $5n^2+3n+1 \geq 1n^2$ when $n \geq 0$

  So $n \geq 0$ then $5n^2+3n+1 \geq 1n^2$

- From the definition **$f(n) \geq c*g(n)$ for $n \geq 0$**, Here c=1, n_0=0 and **$g(n)=n^2$**
- **Hence, $5n2+3n+1 = \Omega(n^2)$ proved.**

Cont...

3. Theta Notation (Tight bound):

- It is denoted by Θ.
- By this method, the running time is between upper bound and lower bound.

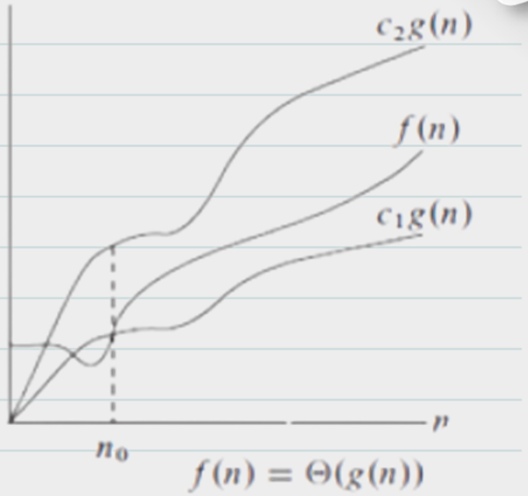
Definition:

- Let f(n) and g(n) be two non-negative functions.
- The function **$f(n)=\Theta(g(n))$** if there exist positive constants c1, c2 and n_0 such that,

  $c1*g(n) \leq \mathbf{f(n)} \leq \mathbf{c2*g(n)}$ for all $n \geq n_0$.
- Then we can say that $f(n) \varepsilon \Theta (g(n))$

Cont...

- Theta (Θ) notation gives bounds for a function to within a constant factor.
- We write **f(n)=Θ(g(n))** if there are positive constants n_0, c1 and c2, such that to the right of n_0, the value of **f(n)** is always lie between **c1g(n)** and **c2g(n)** inclusive.



Cont...

Example - Prove that $5n2+3n+1 = \Theta(n^2)$

- **Solution -** Let $f(n)= 5n^2+3n+1$ and $g(n)=n^2$

  $c1*g(n) \leq f(n) \leq c2*g(n)$

  $1n^2 \leq 5n^2+3n+1 \leq 9n^2$ when $n \geq 1$

  So $n \geq 1$ then $1n^2 \leq 5n^2+3n+1 \leq 9n^2$

- From the definition $c1*g(n) \leq f(n) \leq c2*g(n)$ **for n** ≥ 1, Here $c1=1$, $c2=9$ $n_0=1$ and **$g(n)=n^2$**
- **Hence, $5n2+3n+1 = \Theta(n^2)$ proved.**

Cont...

Properties of order of growth:

1. if $f_1(n)$ is order of $g_1(n)$ and $f_2(n)$ is order of $g_2(n)$, then $f_1(n)+f_2(n) \in O(max(g_1(n),g_2(n))$.
2. Polynomials of degree m $\in \Theta(n^m)$ maximum degree is considered from the polynomial.

For Eg: $a_1n^3+a_2n^2+a_3n+c$ has the order of growth $\Theta(n^3)$.

3. $O(1) < O(\log^n) < O(\sqrt{n}) < O(n) < O(n\log^n) < O(n^2) < O(n^3) < \ldots\ldots < O(2^n) < O(3^n) < \ldots\ldots < O(n^n)$.
4. Exponential function a^n have different orders of growth for different values of a.

Cont...

Key points:

- O(g(n)) is a class of functions, f(n) that grows less fast than g(n), that means f(n) possess the time complexity which is always lesser than the time complexities of g(n).
- Θ(g(n)) is a class of functions, f(n) that grows at same rate as g(n).
- Ω(g(n))is a class of functions, f(n) that grows faster than or atleast as fast as g(n).

Cont...

Assignment 1

Consider the following two functions

$$g1(n) = \begin{cases} n^3, \text{ for } 0 \le n \le 10{,}000 \\ n^2, \text{ for } n > 10{,}000 \end{cases} \qquad g2(n) = \begin{cases} n, \text{ for } 0 \le n \le 100 \\ n^3, \text{ for } n > 100 \end{cases}$$

Which one of the following are true?

a) g1(n) is $O(g2(n))$

b) g1(n) is $O(n^3)$

c) g2(n) is $O(g1(n))$

d) g2(n) is $O(n)$

Cont...

Assignment 2

Which of the given options provides the increasing order of asymptotic complexity of function f1, f2, f3, f4? where

- $f1(n)=2^n$
- $f2(n)=n^{3/2}$
- $f3(n)=n\log n$
- $f4(n)=n^{\log n}$

a) f3, f2, f4, f1

b) f3, f2, f1, f4

c) f2, f3, f1, f4

d) f2, f3, f4, f1


END
Thank You!